Aniog, Dayer C.
BSIT - 303I

1. Problem 1: Student Grade Analysis System

Source code:

```
students = [
    ("Ana", (85, 90, 88)),
    ("Ben", (78, 82, 88)),
    ("Carlo", (92, 95, 91)),
    ("Eric", (70, 75, 72)),
    ("Faith", (95, 93, 97))
]

print("STUDENT GRADE REPORT")

averages = []


for name, grades in students:
    average = sum(grades) / len(grades)
    averages.append((name, average))
    print(f"{name}: Average = {average:.2f}")


top_student = max(averages, key=lambda student: student[1])

print(f"\nTop Student: {top_student[0]}")


qualified = []

for name, average in averages:
    if average >= 85:
        qualified.append(name)

print("Qualified Students:", qualified)


averages.sort(key=lambda student: student[1], reverse=True)

print("\nSorted Results:")
print(averages)
```

OUTPUT:

```
STUDENT GRADE REPORT
Ana: Average = 87.67
Ben: Average = 82.67
Carlo: Average = 92.67
Eric: Average = 72.33
Faith: Average = 95.00

Top Student: Faith
Qualified Students: ['Ana', 'Carlo', 'Faith']

Sorted Results:
[('Faith', 95.0), ('Carlo', 92.66666666666667), ('Ana', 87.66666666666667), ('Ben', 82.66666666666667), ('Eric', 72.33333333333333)]
```

2. Problem 2: Duplicate Removal Without Losing Order

Source code:

```python
numbers = [
    12, 5, 8, 12, 3, 5, 9, 8,
    15, 3, 20, 12, 9, 25, 15
]

def analyze_numbers(numbers):
    unique = []
    frequency = {}

    for num in numbers:
        if num not in unique:
            unique.append(num)

    for num in numbers:
        if num in frequency:
            frequency[num] += 1
        else:
            frequency[num] = 1

    highest = max(frequency.values())

    most_frequent = []

    for num in unique:
        if frequency[num] == highest:
            most_frequent.append(num)

    unique_tuple = tuple(unique)

    print("===== DUPLICATE REMOVAL REPORT =====")

    print("\nOriginal List:")
    print(numbers)

    print("\nUnique List:")
    print(unique)

    print("\nFrequency Report:")
    for num, count in frequency.items():
        print(f"{num}: {count} time(s)")

    print("\nMost Frequent Number(s):")
    print(most_frequent)

    print("\nHighest Frequency:")
    print(highest)

    print("\nUnique Tuple:")
    print(unique_tuple)


analyze_numbers(numbers)

print("\n===== ADDITIONAL TEST CASE =====")
analyze_numbers([4, 7, 4, 7, 9, 2, 9])
```

OUTPUT:

```
===== DUPLICATE REMOVAL REPORT =====

Original List:
[12, 5, 8, 12, 3, 5, 9, 8, 15, 3, 20, 12, 9, 25, 15]

Unique List:
[12, 5, 8, 3, 9, 15, 20, 25]

Frequency Report:
12: 3 time(s)
5: 2 time(s)
8: 2 time(s)
3: 2 time(s)
9: 2 time(s)
15: 2 time(s)
20: 1 time(s)
25: 1 time(s)

Most Frequent Number(s):
[12]

Highest Frequency:
3

Unique Tuple:
(12, 5, 8, 3, 9, 15, 20, 25)

===== ADDITIONAL TEST CASE =====
===== DUPLICATE REMOVAL REPORT =====

Original List:
[4, 7, 4, 7, 9, 2, 9]

Unique List:
[4, 7, 9, 2]

Frequency Report:
4: 2 time(s)
7: 2 time(s)
9: 2 time(s)
2: 1 time(s)

Most Frequent Number(s):
[4, 7, 9]

Highest Frequency:
2

Unique Tuple:
(4, 7, 9, 2)
```

3. Problem 3: Tuple-Based Inventory Management

Source Code:

```
inventory = [
    ("KB101", "Keyboard", 25, 450.00),
    ("MS102", "Mouse", 40, 250.00),
    ("HD103", "Hard Drive", 12, 2800.00),
    ("MN104", "Monitor", 8, 6500.00),
    ("US105", "USB Drive", 30, 350.00)
]

def analyze_inventory(inventory):
    results = []

    for code, name, quantity, price in inventory:
        total = quantity * price
        results.append((code, name, quantity, price, total))

    print("\n===== INVENTORY MANAGEMENT SYSTEM =====")

    print("\nProduct Inventory:")
    for code, name, quantity, price, total in results:
        print(f"{code} - {name}")
        print(f"Quantity: {quantity}")
        print(f"Unit Price: {price:.2f}")
        print(f"Total Value: {total:.2f}\n")

    total_inventory = sum(item[4] for item in results)
    print(f"Total Inventory Value: {total_inventory:.2f}")

    print("\nProducts with Fewer Than 15 Units:")
    for item in results:
        if item[2] < 15:
            print(f"{item[1]} - {item[2]} units")

    highest_value = max(item[4] for item in results)

    print("\nProduct(s) with Highest Inventory Value:")
    for item in results:
        if item[4] == highest_value:
            print(f"{item[1]} - {item[4]:.2f}")

    sorted_inventory = sorted(
        results, key=lambda item: item[3], reverse=True
    )

    print("\nInventory Sorted by Unit Price:")
    for item in sorted_inventory:
        print(f"{item[1]} - {item[3]:.2f}")

    print("\nProduct Summary:")
    for code, name, quantity, price, total in results:
        print((code, name, quantity, total))

    return results


results = analyze_inventory(inventory)

print("\n===== SEARCH PRODUCT =====")
search_code = input("Enter Product Code: ").strip().upper()

found = False

for code, name, quantity, price, total in results:
    if code == search_code:
        print("\nProduct Found!")
        print(f"Product Code: {code}")
        print(f"Product Name: {name}")
        print(f"Quantity: {quantity}")
        print(f"Unit Price: {price:.2f}")
        print(f"Total Value: {total:.2f}")
        found = True
        break

if not found:
    print("Product not found.")

print("\n===== ADDITIONAL TEST CASE =====")

test_inventory = [
    ("PR101", "Printer", 5, 4500.00),
    ("SP102", "Speaker", 20, 1200.00),
    ("CM103", "Camera", 10, 3000.00)
]

analyze_inventory(test_inventory)
```

Output:

```
===== INVENTORY MANAGEMENT SYSTEM =====

Product Inventory:
KB101 - Keyboard
Quantity: 25
Unit Price: 450.00
Total Value: 11250.00

MS102 - Mouse
Quantity: 40
Unit Price: 250.00
Total Value: 10000.00

HD103 - Hard Drive
Quantity: 12
Unit Price: 2800.00
Total Value: 33600.00

MN104 - Monitor
Quantity: 8
Unit Price: 6500.00
Total Value: 52000.00

US105 - USB Drive
Quantity: 30
Unit Price: 350.00
Total Value: 10500.00

Total Inventory Value: 117350.00

Products with Fewer Than 15 Units:
Hard Drive - 12 units
Monitor - 8 units

Product(s) with Highest Inventory Value:
Monitor - 52000.00

Inventory Sorted by Unit Price:
Monitor - 6500.00
Hard Drive - 2800.00
Keyboard - 450.00
USB Drive - 350.00
Mouse - 250.00

Product Summary:
('KB101', 'Keyboard', 25, 11250.0)
('MS102', 'Mouse', 40, 10000.0)
('HD103', 'Hard Drive', 12, 33600.0)
('MN104', 'Monitor', 8, 52000.0)
('US105', 'USB Drive', 30, 10500.0)
```

```
    ===== SEARCH PRODUCT =====
... Enter Product Code: KB101

    Product Found!
    Product Code: KB101
    Product Name: Keyboard
    Quantity: 25
    Unit Price: 450.00
    Total Value: 11250.00

    ===== ADDITIONAL TEST CASE =====

    ===== INVENTORY MANAGEMENT SYSTEM =====

    Product Inventory:
    PR101 - Printer
    Quantity: 5
    Unit Price: 4500.00
    Total Value: 22500.00

    SP102 - Speaker
    Quantity: 20
    Unit Price: 1200.00
    Total Value: 24000.00

    CM103 - Camera
    Quantity: 10
    Unit Price: 3000.00
    Total Value: 30000.00

    Total Inventory Value: 76500.00

    Products with Fewer Than 15 Units:
    Printer - 5 units
    Camera - 10 units

    Product(s) with Highest Inventory Value:
    Camera - 30000.00

    Inventory Sorted by Unit Price:
    Printer - 4500.00
    Camera - 3000.00
    Speaker - 1200.00

    Product Summary:
    ('PR101', 'Printer', 5, 22500.0)
    ('SP102', 'Speaker', 20, 24000.0)
    ('CM103', 'Camera', 10, 30000.0)
    [('PR101', 'Printer', 5, 4500.0, 22500.0),
     ('SP102', 'Speaker', 20, 1200.0, 24000.0),
     ('CM103', 'Camera', 10, 3000.0, 30000.0)]
```

4. **Problem 4: Tuple-Based Employee Payroll Analysis**
   Source code:

```python
#PROBLEM 4
employees = [
    ("E001", "Alice", 40, 250),
    ("E002", "Brian", 45, 300),
    ("E003", "Clara", 38, 280),
    ("E004", "David", 50, 350),
    ("E005", "Ella", 42, 320)
]


def calculate_payroll(employees):
    payroll = []

    for emp_id, name, hours, rate in employees:
        regular_hours = min(hours, 40)
        overtime_hours = max(hours - 40, 0)

        regular_pay = regular_hours * rate
        overtime_pay = overtime_hours * rate * 1.5
        gross_salary = regular_pay + overtime_pay

        payroll.append((
            emp_id, name, hours, rate,
            regular_pay, overtime_pay, gross_salary
        ))

    return payroll


def display_report(payroll):
    print("\n========== EMPLOYEE PAYROLL REPORT ==========")

    for emp in payroll:
        emp_id, name, hours, rate, regular, overtime, gross = emp

        print(f"\nEmployee ID: {emp_id}")
        print(f"Name: {name}")
        print(f"Hours Worked: {hours}")
        print(f"Hourly Rate: PHP {rate:,.2f}")
        print(f"Regular Pay: PHP {regular:,.2f}")
        print(f"Overtime Pay: PHP {overtime:,.2f}")
        print(f"Gross Salary: PHP {gross:,.2f}")

    print("\n========== OVERTIME EMPLOYEES ==========")

    for emp in payroll:
        if emp[2] > 40:
            print(f"{emp[1]} - {emp[2] - 40} overtime hours")

    highest = max(payroll, key=lambda emp: emp[6])

    print("\n========== HIGHEST GROSS SALARY ==========")
    print(f"Employee: {highest[1]}")
    print(f"Gross Salary: PHP {highest[6]:,.2f}")

    total_salary = sum(emp[6] for emp in payroll)
    average_salary = total_salary / len(payroll)
```

```
    print("\n========== AVERAGE GROSS SALARY ==========")
    print(f"Average: PHP {average_salary:,.2f}")

    sorted_payroll = sorted(
        payroll, key=lambda emp: emp[6], reverse=True
    )

    print("\n========== SALARY RANKING ==========")

    for rank, emp in enumerate(sorted_payroll, 1):
        print(f"{rank}. {emp[1]} - PHP {emp[6]:,.2f}")

    print("\n========== PAYROLL TUPLES ==========")

    for emp in payroll:
        print(emp)


def search_employee(payroll):
    print("\n========== EMPLOYEE SEARCH ==========")

    search_id = input("Enter Employee ID: ").strip().upper()

    for emp in payroll:
        if emp[0] == search_id:
            print("\nEmployee Found!")
            print(f"Employee ID: {emp[0]}")
            print(f"Name: {emp[1]}")
            print(f"Hours Worked: {emp[2]}")
            print(f"Hourly Rate: PHP {emp[3]:,.2f}")
            print(f"Regular Pay: PHP {emp[4]:,.2f}")
            print(f"Overtime Pay: PHP {emp[5]:,.2f}")
            print(f"Gross Salary: PHP {emp[6]:,.2f}")
            return

    print("Employee not found.")


payroll = calculate_payroll(employees)

display_report(payroll)
search_employee(payroll)

print("\n========== ADDITIONAL TEST CASE ==========")

test_employees = [
    ("T001", "John", 40, 200),
    ("T002", "Mark", 48, 250)
]

test_payroll = calculate_payroll(test_employees)
display_report(test_payroll)
```

OUTPUT:

```
...
    ========== EMPLOYEE PAYROLL REPORT ==========

    Employee ID: E001
    Name: Alice
    Hours Worked: 40
    Hourly Rate: PHP 250.00
    Regular Pay: PHP 10,000.00
    Overtime Pay: PHP 0.00
    Gross Salary: PHP 10,000.00

    Employee ID: E002
    Name: Brian
    Hours Worked: 45
    Hourly Rate: PHP 300.00
    Regular Pay: PHP 12,000.00
    Overtime Pay: PHP 2,250.00
    Gross Salary: PHP 14,250.00

    Employee ID: E003
    Name: Clara
    Hours Worked: 38
    Hourly Rate: PHP 280.00
    Regular Pay: PHP 10,640.00
    Overtime Pay: PHP 0.00
    Gross Salary: PHP 10,640.00

    Employee ID: E004
    Name: David
    Hours Worked: 50
    Hourly Rate: PHP 350.00
    Regular Pay: PHP 14,000.00
    Overtime Pay: PHP 5,250.00
    Gross Salary: PHP 19,250.00

    Employee ID: E005
    Name: Ella
    Hours Worked: 42
    Hourly Rate: PHP 320.00
    Regular Pay: PHP 12,800.00
    Overtime Pay: PHP 960.00
    Gross Salary: PHP 13,760.00

    ========== OVERTIME EMPLOYEES ==========
    Brian - 5 overtime hours
    David - 10 overtime hours
    Ella - 2 overtime hours

    ========== HIGHEST GROSS SALARY ==========
    Employee: David
    Gross Salary: PHP 19,250.00

    ========== AVERAGE GROSS SALARY ==========
    Average: PHP 13,580.00

    ========== SALARY RANKING ==========
    1. David - PHP 19,250.00
    2. Brian - PHP 14,250.00
```

5. Problem 5: Leaderboard Ranking System
Source Code:

```
#PROBLEM 5
contestants = [
    ("C001", "Alex", (85, 90, 88)),
    ("C002", "Bea", (95, 88, 92)),
    ("C003", "Chris", (90, 90, 90)),
    ("C004", "Dana", (95, 85, 87)),
    ("C005", "Evan", (88, 92, 91)),
    ("C006", "Gia", (95, 85, 87))
]


def calculate_scores(contestants):
    leaderboard = []

    for contestant_id, name, scores in contestants:
        total = sum(scores)
        average = total / len(scores)
        highest_score = max(scores)

        leaderboard.append((
            contestant_id,
            name,
            scores,
            total,
            average,
            highest_score
        ))

    return leaderboard


def rank_contestants(leaderboard):
    return sorted(
        leaderboard,
        key=lambda contestant: (
            -contestant[3],
            -contestant[5]
        )
    )


def display_leaderboard(leaderboard):
    print("\n========== COMPETITION LEADERBOARD ==========")

    for rank, contestant in enumerate(leaderboard, 1):
        contestant_id, name, scores, total, average, highest = contestant

        print(f"\nRank: {rank}")
        print(f"Contestant ID: {contestant_id}")
        print(f"Name: {name}")
        print(f"Problem Scores: {scores}")
        print(f"Total Score: {total}")
        print(f"Average Score: {average:.2f}")
        print(f"Highest Individual Score: {highest}")

    print("\n========== TOP 3 CONTESTANTS ==========")

    for rank, contestant in enumerate(leaderboard[:3], 1):
```

```
    for rank, contestant in enumerate(leaderboard, 1):
        contestant_id, name, scores, total, average, highest = contestant

        print(f"\nRank: {rank}")
        print(f"Contestant ID: {contestant_id}")
        print(f"Name: {name}")
        print(f"Problem Scores: {scores}")
        print(f"Total Score: {total}")
        print(f"Average Score: {average:.2f}")
        print(f"Highest Individual Score: {highest}")

    print("\n========== TOP 3 CONTESTANTS ==========")

    for rank, contestant in enumerate(leaderboard[:3], 1):
        print(
            f"{rank}. {contestant[1]} - "
            f"Total: {contestant[3]} | "
            f"Average: {contestant[4]:.2f}"
        )

    print("\n========== QUALIFIED CONTESTANTS ==========")
    print("Average Score of At Least 90:")

    qualified = [
        contestant for contestant in leaderboard
        if contestant[4] >= 90
    ]

    if qualified:
        for contestant in qualified:
            print(
                f"{contestant[1]} - "
                f"Average: {contestant[4]:.2f}"
            )
    else:
        print("No qualified contestants.")

    print("\n========== LEADERBOARD TUPLES ==========")

    for contestant in leaderboard:
        print(contestant)


scores = calculate_scores(contestants)
leaderboard = rank_contestants(scores)

display_leaderboard(leaderboard)


print("\n========== ADDITIONAL TEST CASE ==========")

test_contestants = [
    ("T001", "Mia", (90, 90, 90)),
    ("T002", "Noah", (95, 85, 90)),
    ("T003", "Lia", (95, 90, 85))
]

test_scores = calculate_scores(test_contestants)
test_leaderboard = rank_contestants(test_scores)

display_leaderboard(test_leaderboard)
```

Output:

```
...
---------- COMPETITION LEADERBOARD ----------

Rank: 1
Contestant ID: C002
Name: Bea
Problem Scores: (95, 88, 92)
Total Score: 275
Average Score: 91.67
Highest Individual Score: 95

Rank: 2
Contestant ID: C005
Name: Evan
Problem Scores: (88, 92, 91)
Total Score: 271
Average Score: 90.33
Highest Individual Score: 92

Rank: 3
Contestant ID: C003
Name: Chris
Problem Scores: (90, 90, 90)
Total Score: 270
Average Score: 90.00
Highest Individual Score: 90

Rank: 4
Contestant ID: C004
Name: Dana
Problem Scores: (95, 85, 87)
Total Score: 267
Average Score: 89.00
Highest Individual Score: 95

Rank: 5
Contestant ID: C006
Name: Gia
Problem Scores: (95, 85, 87)
Total Score: 267
Average Score: 89.00
Highest Individual Score: 95

Rank: 6
Contestant ID: C001
Name: Alex
Problem Scores: (85, 90, 88)
Total Score: 263
Average Score: 87.67
Highest Individual Score: 90

---------- TOP 3 CONTESTANTS ----------
1. Bea - Total: 275 | Average: 91.67
2. Evan - Total: 271 | Average: 90.33
3. Chris - Total: 270 | Average: 90.00

---------- QUALIFIED CONTESTANTS ----------
Average Score of At Least 90:
```

```
---------- QUALIFIED CONTESTANTS ----------
Average Score of At Least 90:
Bea - Average: 91.67
Evan - Average: 90.33
Chris - Average: 90.00

---------- LEADERBOARD TUPLES ----------
('C002', 'Bea', (95, 88, 92), 275, 91.66666666666667, 95)
('C005', 'Evan', (88, 92, 91), 271, 90.33333333333333, 92)
('C003', 'Chris', (90, 90, 90), 270, 90.0, 90)
('C004', 'Dana', (95, 85, 87), 267, 89.0, 95)
('C006', 'Gia', (95, 85, 87), 267, 89.0, 95)
('C001', 'Alex', (85, 90, 88), 263, 87.66666666666667, 90)

---------- ADDITIONAL TEST CASE ----------

---------- COMPETITION LEADERBOARD ----------

Rank: 1
Contestant ID: T002
Name: Noah
Problem Scores: (95, 85, 90)
Total Score: 270
Average Score: 90.00
Highest Individual Score: 95

Rank: 2
Contestant ID: T003
Name: Lia
Problem Scores: (95, 90, 85)
Total Score: 270
Average Score: 90.00
Highest Individual Score: 95

Rank: 3
Contestant ID: T001
Name: Mia
Problem Scores: (90, 90, 90)
Total Score: 270
Average Score: 90.00
Highest Individual Score: 90

---------- TOP 3 CONTESTANTS ----------
1. Noah - Total: 270 | Average: 90.00
2. Lia - Total: 270 | Average: 90.00
3. Mia - Total: 270 | Average: 90.00

---------- QUALIFIED CONTESTANTS ----------
Average Score of At Least 90:
Noah - Average: 90.00
Lia - Average: 90.00
Mia - Average: 90.00

---------- LEADERBOARD TUPLES ----------
('T002', 'Noah', (95, 85, 90), 270, 90.0, 95)
('T003', 'Lia', (95, 90, 85), 270, 90.0, 95)
('T001', 'Mia', (90, 90, 90), 270, 90.0, 90)
```

6. **Problem 6: Advanced List and Tuple Data Analyzer**
   Source Code:

```
data = [
    14, 22, 18, 14, 35, 27, 22, 40,
    18, 14, 31, 27, 45, 22, 35, 50,
    18, 40, 27, 31
]


def display_original(data):
    print("\n========== ORIGINAL DATA ==========")
    print(data)


def display_sorted(data):
    print("\n========== SORTED DATA ==========")
    print(sorted(data))


def remove_duplicates(data):
    unique = []

    for num in data:
        if num not in unique:
            unique.append(num)

    return unique


def display_frequency(data):
    print("\n========== FREQUENCY TABLE ==========")

    frequency = {}

    for num in data:
        if num in frequency:
            frequency[num] += 1
        else:
            frequency[num] = 1

    for num, count in frequency.items():
        print(f"{num}: {count} time(s)")

    highest = max(frequency.values())

    print("\nMost Frequent Value(s):")

    for num, count in frequency.items():
        if count == highest:
            print(f"{num} - {count} time(s)")
```

```
            print(f"{num} - {count} time(s)")


def display_min_max(data):
    print("\n========== MINIMUM AND MAXIMUM ==========")
    print(f"Minimum Value: {min(data)}")
    print(f"Maximum Value: {max(data)}")


def calculate_mean(data):
    return sum(data) / len(data)


def calculate_median(data):
    sorted_data = sorted(data)
    length = len(sorted_data)
    middle = length // 2

    if length % 2 == 0:
        median = (
            sorted_data[middle - 1] +
            sorted_data[middle]
        ) / 2
    else:
        median = sorted_data[middle]

    return median


def display_mean_median(data):
    print("\n========== MEAN AND MEDIAN ==========")
    print(f"Mean: {calculate_mean(data):.2f}")
    print(f"Median: {calculate_median(data):.2f}")


def display_second_largest(data):
    print("\n========== SECOND-LARGEST VALUE ==========")

    unique = remove_duplicates(data)
    sorted_unique = sorted(unique, reverse=True)

    if len(sorted_unique) >= 2:
        print(f"Second-Largest Distinct Value: {sorted_unique[1]}")
    else:
        print("No second-largest distinct value.")


def convert_to_tuple(data):
    print("\n========== TUPLE CONVERSION ==========")

    converted_tuple = tuple(data)
```

```
    converted_tuple = tuple(data)

    print("Converted Tuple:")
    print(converted_tuple)


def search_value(data):
    print("\n========== SEARCH VALUE ==========")

    try:
        search = int(input("Enter a number to search: "))

        if search in data:
            count = data.count(search)

            print(f"\nNumber {search} was found!")
            print(f"Number of Occurrences: {count}")
        else:
            print(f"\nNumber {search} was not found.")

    except ValueError:
        print("Invalid input! Please enter a number.")


def display_menu():
    print("\n========== DATA ANALYZER ==========")
    print("1. Display Original Data")
    print("2. Display Sorted Data")
    print("3. Remove Duplicates")
    print("4. Display Frequency Table")
    print("5. Display Minimum and Maximum")
    print("6. Display Mean and Median")
    print("7. Display Second-Largest Distinct Value")
    print("8. Convert Data into a Tuple")
    print("9. Search for a Value")
    print("10. Exit")
    print("==================================")


def run_analyzer(data):
    while True:
        display_menu()

        try:
            choice = int(input("\nEnter your choice (1-10): "))

            if choice == 1:
                display_original(data)

            elif choice == 2:
                display_sorted(data)
```

```python
def run_analyzer(data):
    while True:
        display_menu()

        try:
            choice = int(input("\nEnter your choice (1-10): "))

            if choice == 1:
                display_original(data)

            elif choice == 2:
                display_sorted(data)

            elif choice == 3:
                print("\n========== UNIQUE DATA ==========")
                print(remove_duplicates(data))

            elif choice == 4:
                display_frequency(data)

            elif choice == 5:
                display_min_max(data)

            elif choice == 6:
                display_mean_median(data)

            elif choice == 7:
                display_second_largest(data)

            elif choice == 8:
                convert_to_tuple(data)

            elif choice == 9:
                search_value(data)

            elif choice == 10:
                print("\nThank you for using Data Analyzer!")
                break

            else:
                print("\nInvalid choice! Please select 1-10.")

        except ValueError:
            print("\nInvalid input! Please enter a valid number.")


def additional_test():
    test_data = [10, 20, 10, 30, 20, 40, 30, 40]

    print("\n========== ADDITIONAL TEST CASE ==========")
    print("Original Data:", test_data)
    print("Sorted Data:", sorted(test_data))
    print("Unique Data:", remove_duplicates(test_data))
    print("Mean:", calculate_mean(test_data))
    print("Median:", calculate_median(test_data))
    display_frequency(test_data)
    display_second_largest(test_data)
    print("Converted Tuple:", tuple(test_data))


run_analyzer(data)
additional_test()
```

Output:

```
========== DATA ANALYZER ==========
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
==================================

Enter your choice (1-10): 1

========== ORIGINAL DATA ==========
[14, 22, 18, 14, 35, 27, 22, 40, 18, 14, 31, 27, 45, 22, 35, 50, 18, 40, 27, 31]

========== DATA ANALYZER ==========
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
==================================

Enter your choice (1-10): 2

========== SORTED DATA ==========
[14, 14, 14, 18, 18, 18, 22, 22, 22, 27, 27, 27, 31, 31, 35, 35, 40, 40, 45, 50]
```

```
========== DATA ANALYZER ==========
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
==================================

Enter your choice (1-10): 3

========== UNIQUE DATA ==========
[14, 22, 18, 35, 27, 40, 31, 45, 50]
```

```
...
    Enter your choice (1-10): 4

    ========== FREQUENCY TABLE ==========
    14: 3 time(s)
    22: 3 time(s)
    18: 3 time(s)
    35: 2 time(s)
    27: 3 time(s)
    40: 2 time(s)
    31: 2 time(s)
    45: 1 time(s)
    50: 1 time(s)

    Most Frequent Value(s):
    14 - 3 time(s)
    22 - 3 time(s)
    18 - 3 time(s)
    27 - 3 time(s)

    ========== DATA ANALYZER ==========
    1. Display Original Data
    2. Display Sorted Data
    3. Remove Duplicates
    4. Display Frequency Table
    5. Display Minimum and Maximum
    6. Display Mean and Median
    7. Display Second-Largest Distinct Value
    8. Convert Data into a Tuple
    9. Search for a Value
    10. Exit
    ==================================

    Enter your choice (1-10): 5

    ========== MINIMUM AND MAXIMUM ==========
    Minimum Value: 14
    Maximum Value: 50
```

```
========== DATA ANALYZER ==========
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
==================================

Enter your choice (1-10): 6

========== MEAN AND MEDIAN ==========
Mean: 27.50
Median: 27.00

========== DATA ANALYZER ==========
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
==================================

Enter your choice (1-10): 7

========== SECOND-LARGEST VALUE ==========
Second-Largest Distinct Value: 45
```

```
========== SECOND-LARGEST VALUE ==========
Second-Largest Distinct Value: 45

========== DATA ANALYZER ==========
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
==================================

Enter your choice (1-10): 8

========== TUPLE CONVERSION ==========
Converted Tuple:
(14, 22, 18, 14, 35, 27, 22, 40, 18, 14, 31, 27, 45, 22, 35, 50, 18, 40, 27, 31)

========== DATA ANALYZER ==========
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
==================================

Enter your choice (1-10): 9


Enter your choice (1-10): 10

Thank you for using Data Analyzer!

========== ADDITIONAL TEST CASE ==========
Original Data: [10, 20, 10, 30, 20, 40, 30, 40]
Sorted Data: [10, 10, 20, 20, 30, 30, 40, 40]
Unique Data: [10, 20, 30, 40]
Mean: 25.0
Median: 25.0

========== FREQUENCY TABLE ==========
10: 2 time(s)
20: 2 time(s)
30: 2 time(s)
40: 2 time(s)

Most Frequent Value(s):
10 - 2 time(s)
20 - 2 time(s)
30 - 2 time(s)
40 - 2 time(s)

========== SECOND-LARGEST VALUE ==========
Second-Largest Distinct Value: 30
Converted Tuple: (10, 20, 10, 30, 20, 40, 30, 40)
```